

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Sergiu Gherasim, FAF-242

Report

Laboratory Work No. 5

Operating Systems

Topic: The File System with Your Own Hands

Inodes, Links, Permissions and Disks on Linux

Checked by:
Patricia Reiţman, *university assistant*

Chişinău – 2026

CONTENTS

Introduction **3**

1 Short Theory **4**
1.1 Files, Names and Inodes 4
1.2 Hard and Symbolic Links 4
1.3 Permissions 4
1.4 Disks and Pseudo File Systems 4

2 The Tree and the File Types **5**

3 Inodes and Links **7**
3.1 An Inode, and a Name that Is Not Part of It 7
3.2 Hard Link, then Remove the "Original" 7
3.3 Symbolic Link, then Remove Its Target 7
3.4 Directories Have Links Too 7
3.5 Size Is Not Space 8

4 Permissions **10**
4.1 Files 10
4.2 Directories 10
4.3 Special Bits and Other People's Files 10

5 Disks and the Kernel as Files **12**

6 Stretch Tasks **14**
6.1 (a) Magic Numbers 14
6.2 (c) Deleting a File That Is Still Open 14

Conclusions **15**

INTRODUCTION

The purpose of this laboratory work is to check, on a real Linux system, every idea from the file system lecture: the directory tree and the file types, inodes standing behind the names, hard and symbolic links, permission bits, and finally the disks and pseudo file systems underneath. No programming is required; the work consists of commands, careful observation of their output, and short explanations.

All experiments were done on Ubuntu 24.04 LTS (running in a virtual machine on my laptop), as the normal user `sergiu` and not as root, because root ignores most permission checks and the experiments in Part 3 would give misleading results.

```
$ whoami
sergiu
$ mkdir -p ~/os-lab5 && cd ~/os-lab5
```

Listing 1: Preparing the working directory

1 SHORT THEORY

1.1 Files, Names and Inodes

In a Unix file system the name of a file and the file itself are two different objects. The file is described by an *inode*: a fixed-size record that stores the type, the owner, the permission bits, the size, the timestamps, the number of names (links) and the addresses of the data blocks. The inode does **not** store the name. Names live in directories, and a directory is just a table of pairs *(name, inode number)*. Opening a path therefore means walking the tree from `/`, looking every component up in the table of its parent directory.

1.2 Hard and Symbolic Links

A *hard link* is a second entry in some directory that points to the same inode number. All hard links are equal; there is no "original". The inode keeps a link counter and the kernel frees the inode and its blocks only when this counter reaches zero and no process keeps the file open. A *symbolic link* is a separate small file of type `l` whose content is a path. The kernel follows that path when the link is opened, so a symbolic link can point to another file system, to a directory, or to nothing at all (a dangling link).

1.3 Permissions

Every inode has an owner, a group and nine permission bits: read, write and execute for the owner (`u`), the group (`g`) and the others (`o`). The bits are often written in octal, one digit per class ($r = 4$, $w = 2$, $x = 1$). On a directory, `r` allows listing the names, `w` allows creating and deleting entries, and `x` allows passing through the directory to reach the inodes behind the names. Three extra bits exist: *setuid* (`s`, the program runs with the identity of the file owner), *setgid* and the *sticky* bit (`t`, on a shared directory only the owner of a file may delete it). New files get the permissions requested by the program, minus the bits set in the process *umask*.

1.4 Disks and Pseudo File Systems

A disk is a block device that is split into partitions; a file system is created inside a partition and *mounted* onto a directory of the single tree. Not every mounted file system lives on a disk: `/proc` and `/sys` are generated by the kernel on the fly and expose processes, memory and hardware as ordinary-looking text files.

2 THE TREE AND THE FILE TYPES

```
$ ls /
bin                boot  etc   lib                media  opt   root  sbin                snap  sys  usr
bin.usr-is-merged  dev   home  lib.usr-is-merged  mnt    proc  run   sbin.usr-is-merged  srv   tmp  var
$ ls -ld /etc /etc/passwd /bin /dev/null /tmp
lrwxrwxrwx   1 root root    7 Apr 22  2024 /bin -> usr/bin
crw-rw-rw-   1 root root 1, 3 Oct 25 18:02 /dev/null
drwxr-xr-x 132 root root 4096 Oct 25 18:05 /etc
-rw-r--r--   1 root root 2914 Sep 30 21:14 /etc/passwd
drwxrwxrwt  15 root root 4096 Oct 25 18:40 /tmp
```

Listing 2: The root directory and the types of some important paths

```
$ ls -l /dev | head
total 0
crw-r--r--  1 root root     10, 235 Oct 25 18:02 autofs
drwxr-xr-x  2 root root         320 Oct 25 18:02 block
drwxr-xr-x  2 root root          80 Oct 25 18:02 bsg
crw-rw----  1 root disk     10, 234 Oct 25 18:02 btrfs-control
drwxr-xr-x  3 root root          60 Oct 25 18:02 bus
drwxr-xr-x  2 root root        3900 Oct 25 18:02 char
crw--w----  1 root tty        5,   1 Oct 25 18:02 console
lrwxrwxrwx  1 root root          11 Oct 25 18:02 core -> /proc/kcore
drwxr-xr-x  2 root root          60 Oct 25 18:02 cpu
$ ls -l /dev/vda /dev/tty
brw-rw---- 1 root disk 253, 0 Oct 25 18:02 /dev/vda
crw-rw-rw- 1 root tty    5, 0 Oct 25 18:02 /dev/tty
$ file /usr/bin/ls /etc/passwd /bin
/usr/bin/ls: ELF 64-bit LSB pie executable, ARM aarch64, version 1 (SYSV), dynamically linked, interpreter
    /lib/ld-linux-aarch64.so.1, BuildID[sha1]=3f1c6a0e5d0b..., for GNU/Linux 3.7.0, stripped
/etc/passwd: ASCII text
/bin:        symbolic link to usr/bin
$ du -sh /var/log 2>/dev/null ; du -sh /usr 2>/dev/null
187M    /var/log
6.3G    /usr
```

Listing 3: The first entries of /dev, the file command and the disk usage

Observations

1. For each path in the second line, write its type from the first letter of `ls -l`.

Table 1: File types read from the first column of `ls -ld`

Path	Letter	Type
`/etc`	`d`	directory
`/etc/passwd`	`-`	regular file
`/bin`	`l`	symbolic link
`/dev/null`	`c`	character device
`/tmp`	`d`	directory (with the sticky bit `t`)

2. What is `/bin` on your system: a directory or a link, and to where?

On Ubuntu 24.04 `/bin` is not a directory but a symbolic link to `usr/bin` (a relative target, so it resolves to `/usr/bin`). This is the "merged `/usr`" layout: all programs live in `/usr/bin` and the old path is kept only for compatibility. The `file` command confirms it: `/bin: symbolic link to usr/bin`.

3. Find one character device and one block device in `/dev`.

Character device: `/dev/null` (or `/dev/tty`, `/dev/console`), marked with `c`; it is accessed byte by byte and has no fixed size. Block device: `/dev/vda`, the virtual disk, marked with `b`; it is accessed in blocks and can be partitioned and mounted. Instead of a size, `ls` shows the *major, minor* numbers, e.g. `1, 3` for `/dev/null` and `253, 0` for `/dev/vda`; the major number selects the driver in the kernel.

4. Which is bigger on your laptop, `/var/log` or `/usr`, and by roughly how much?

`/usr` is much bigger: about 6.3 GB against 187 MB for `/var/log`, i.e. roughly 35 times bigger. `/usr` holds all installed programs and libraries, while `/var/log` only holds text logs that are rotated and compressed regularly.

3 INODES AND LINKS

3.1 An Inode, and a Name that Is Not Part of It

```
$ echo hello > notes.txt
$ stat notes.txt
  File: notes.txt
  Size: 6               Blocks: 8          IO Block: 4096   regular file
Device: 253,2   Inode: 1311204     Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1000/  sergiu)   Gid: ( 1000/  sergiu)
Access: 2026-10-25 18:12:03.118224610 +0300
Modify: 2026-10-25 18:12:03.118224610 +0300
Change: 2026-10-25 18:12:03.118224610 +0300
 Birth: 2026-10-25 18:12:03.118224610 +0300
$ ls -li notes.txt
1311204 -rw-rw-r-- 1 sergiu sergiu 6 Oct 25 18:12 notes.txt
```

Listing 4: Creating a file and looking at its inode

3.2 Hard Link, then Remove the "Original"

```
$ ln notes.txt hard.txt
$ ls -li notes.txt hard.txt
1311204 -rw-rw-r-- 2 sergiu sergiu 6 Oct 25 18:12 hard.txt
1311204 -rw-rw-r-- 2 sergiu sergiu 6 Oct 25 18:12 notes.txt
$ rm notes.txt
$ cat hard.txt ; ls -li hard.txt
hello
1311204 -rw-rw-r-- 1 sergiu sergiu 6 Oct 25 18:12 hard.txt
```

Listing 5: Two names for one inode

3.3 Symbolic Link, then Remove Its Target

```
$ ln -s hard.txt soft.txt
$ ls -li hard.txt soft.txt
1311204 -rw-rw-r-- 1 sergiu sergiu 6 Oct 25 18:12 hard.txt
1311219 lrwxrwxrwx 1 sergiu sergiu 8 Oct 25 18:14 soft.txt -> hard.txt
$ cat soft.txt
hello
$ rm hard.txt
$ cat soft.txt ; ls -l soft.txt
cat: soft.txt: No such file or directory
lrwxrwxrwx 1 sergiu sergiu 8 Oct 25 18:14 soft.txt -> hard.txt
```

Listing 6: A symbolic link becomes dangling

3.4 Directories Have Links Too

Before running the commands I predicted 2 links for a new directory (its name in the parent and its own `.`) and 4 after creating two subdirectories (each new `..` points back). The output confirmed the prediction:

```
$ mkdir d ; ls -ld d
drwxrwxr-x 2 sergiu sergiu 4096 Oct 25 18:15 d
$ mkdir d/x d/y ; ls -ld d
drwxrwxr-x 4 sergiu sergiu 4096 Oct 25 18:15 d
```

Listing 7: Link count of a directory

3.5 Size Is Not Space

```
$ echo x > one.txt ; ls -l one.txt ; du -h one.txt
-rw-rw-r-- 1 sergiu sergiu 2 Oct 25 18:16 one.txt
4.0K    one.txt
$ truncate -s 1G big.img ; ls -lh big.img ; du -h big.img
-rw-rw-r-- 1 sergiu sergiu 1.0G Oct 25 18:16 big.img
0       big.img
$ rm big.img
```

Listing 8: Apparent size versus allocated space

Observations

1. Which number in `stat` is the inode, and which tells how many names the file has?

The inode number is the field `Inode: 1311204`; `ls -li` prints the same number in its first column. The number of names is `Links: 1`, which `ls -l` shows in the second column.

2. After `rm notes.txt`, why could you still read `hard.txt`? What does `rm` really remove?

`notes.txt` and `hard.txt` were two directory entries pointing to the same inode 1311204, so the link count was 2. `rm` calls `unlink()`: it removes only the *name* from the directory and decrements the link count. The count dropped to 1, so the inode and its data blocks stayed untouched and were still reachable through `hard.txt`. The data is freed only when the link count becomes 0 and no process has the file open.

3. Why did `cat soft.txt` fail at the end, although `soft.txt` still exists? How big is `soft.txt` and why exactly that size?

`soft.txt` has its own inode (1311219) and only stores the path `hard.txt`. When `cat` opens it, the kernel follows that path, but after `rm hard.txt` no such name exists, so `open` fails with `ENOENT` – the link is dangling. Its size is 8 bytes, exactly the number of characters in the target string `hard.txt` (no terminating zero is stored).

4. A new directory has 2 links, and 4 after you create two subdirectories. Explain where each link comes from.

- link 1: the entry `d` in the parent directory `~/os-lab5`;
- link 2: the entry `.` inside `d`, which points to `d` itself;
- links 3 and 4: the entries `..` inside `d/x` and `d/y`, which point back to `d`.

In general a directory has 2 + (number of subdirectories) links.

5. Why does a 2-byte file use 4.0K, and a 1 GB file use 0?

`ls -l` shows the *size* stored in the inode, while `du` shows the space actually allocated. The file system allocates whole blocks; on ext4 a block is 4096 bytes, so even the 2-byte file `x\n` occupies one full 4 KiB block. `truncate -s 1G` only sets the size field and writes no data, so no block is allocated: the file is *sparse*, and reading it returns zeros generated by the kernel. Size and space are two different things.

4 PERMISSIONS

4.1 Files

```
$ echo secret > s.txt
$ chmod 600 s.txt ; ls -l s.txt
-rw------- 1 sergiu sergiu 7 Oct 25 18:20 s.txt
$ chmod 000 s.txt ; cat s.txt
cat: s.txt: Permission denied
$ chmod u+r s.txt ; cat s.txt
secret
$ umask ; touch new.txt ; mkdir newdir ; ls -ld new.txt newdir
0002
-rw-rw-r-- 1 sergiu sergiu    0 Oct 25 18:21 new.txt
drwxrwxr-x 2 sergiu sergiu 4096 Oct 25 18:21 newdir
```

Listing 9: Permission bits on a regular file and the umask

4.2 Directories

Prediction: with only `r` I expected to see the names but not to be able to open the file; with only `x` the opposite. The results:

```
$ mkdir box ; echo hi > box/f
$ chmod u=r,go= box  ; ls box ; cat box/f     # read only
ls: cannot access 'box/f': Permission denied
f
cat: box/f: Permission denied
$ chmod u=x,go= box  ; ls box ; cat box/f     # execute only
ls: cannot open directory 'box': Permission denied
hi
$ chmod 755 box
```

Listing 10: The box experiment

4.3 Special Bits and Other People's Files

```
$ ls -l /usr/bin/passwd ; ls -ld /tmp
-rwsr-xr-x 1 root root 64152 May 30  2024 /usr/bin/passwd
drwxrwxrwt 15 root root 4096 Oct 25 18:40 /tmp
$ cat /etc/shadow
cat: /etc/shadow: Permission denied
$ ls -l /etc/shadow
-rw-r----- 1 root shadow 1203 Sep 30 21:14 /etc/shadow
```

Listing 11: setuid, sticky bit and a protected file

Observations

1. Write 600, 644, 755 and 700 as rwx strings, and give one real use for each.

Table 2: Common permission modes

Octal	String	Typical use
600	`rw----`	private files: `~/.ssh/id_ed25519`, a password file
644	`rw-r-r-`	ordinary documents and configs readable by all: `/etc/passwd`, `report.md`
755	`rwxr-xr-x`	programs and public directories: `/usr/bin/ls`, `/usr`
700	`rwx---`	private directories or scripts: `~/.ssh`, a personal script

2. With `chmod 000` you own the file but cannot read it. Why does ownership not help?

The kernel checks the permission bits of the class the process belongs to, and only that class. Because my UID equals the owner of the file, the *owner* bits are used, and they are `--`; the group and other bits are not consulted at all. Ownership does not give access by itself, it gives the *right to change the permissions*: that is why `chmod u+r` worked without `sudo` and then `cat` succeeded. Only root bypasses these checks.

3. What is your umask, and how does it explain the permissions of `new.txt` and `newdir`?

The umask is `0002`, so the `w` bit for *others* is removed from every new object. `touch` asks for `666` (files are not created executable), $666 \,\&\, \neg 002 = 664$, which gives `rw-rw-r-`. `mkdir` asks for `777`, $777 \,\&\, \neg 002 = 775$, which gives `rwxrwxr-x`. Ubuntu uses `002` because every user has a private group of their own.

4. Explain the box experiment: what does `r` allow on a directory, and what does `x` allow?

A directory is a table of names and inode numbers. `r` allows *reading that table*: with `u=r`, `ls` could print the name `f`, but it could not `stat` it (hence the error for `box/f`), and `cat box/f` failed because it is not allowed to go through the directory to the inode. `x` allows *searching/traversing*: with `u=x`, `ls box` failed because the list of names cannot be read, but `cat box/f` worked because I already knew the name and the kernel was allowed to look it up and reach its inode.

5. What do the `s` in `/usr/bin/passwd` and the `t` in `/tmp` mean, and why does `passwd` need the `s`?

The `s` in the owner execute position is the *setuid* bit: the program runs with the effective UID of the file owner (root), no matter who starts it. `passwd` needs it because it must write the new password hash into `/etc/shadow`, which is `rw-r---` for `root:shadow` – as shown above, a normal user cannot even read it. The program itself makes sure that a user can only change their own password. The `t` in the others execute position of `/tmp` is the *sticky* bit: everyone may create files in `/tmp` (`rwxrwxrwx`), but only the owner of a file (or of the directory, or root) may delete or rename it, so users cannot remove each other's temporary files.

5 DISKS AND THE KERNEL AS FILES

```
$ lsblk
NAME   MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sr0     11:0    1 1024M  0 rom
vda    253:0    0   64G  0 disk
|-vda1 253:1    0    1G  0 part /boot/efi
‘-vda2 253:2    0   63G  0 part /
$ df -h
Filesystem      Size  Used Avail Use% Mounted on
tmpfs           392M  1.5M  390M   1% /run
/dev/vda2        62G   14G   45G  24% /
tmpfs           2.0G     0  2.0G   0% /dev/shm
tmpfs           5.0M  8.0K  5.0M   1% /run/lock
/dev/vda1       1.1G  6.4M  1.1G   1% /boot/efi
tmpfs           392M  132K  392M   1% /run/user/1000
$ mount | grep " / "
/dev/vda2 on / type ext4 (rw,relatime)
$ grep -c processor /proc/cpuinfo
4
$ head -3 /proc/meminfo
MemTotal:        4010236 kB
MemFree:         1873520 kB
MemAvailable:    3120448 kB
$ head -5 /proc/self/status
Name:   head
Umask:  0002
State:  R (running)
Tgid:   5123
Ngid:   0
$ ls -l /proc/self/fd
total 0
lrwx------ 1 sergiu sergiu 64 Oct 25 18:30 0 -> /dev/pts/0
lrwx------ 1 sergiu sergiu 64 Oct 25 18:30 1 -> /dev/pts/0
lrwx------ 1 sergiu sergiu 64 Oct 25 18:30 2 -> /dev/pts/0
lr-x------ 1 sergiu sergiu 64 Oct 25 18:30 3 -> /proc/5131/fd
```

Listing 12: Block devices, mounted file systems and /proc

Observations

1. Draw your disk: disk name, partitions, which one is mounted at / and with what file system type.

```
/dev/vda  (virtual disk, 64 GB)
+----------------------+-----------------------------------------------------+
| vda1   1 GB          | vda2   63 GB                                        |
| vfat (EFI system)    | ext4                                                |
| mounted on /boot/efi | mounted on /   (rw,relatime)                        |
+----------------------+-----------------------------------------------------+
```

Listing 13: Layout of the disk

The single disk is `vda` (major 253). The root file system `/` is the partition `vda2`, formatted as `ext4`; the small `vda1` is the EFI partition. `sr0` is the (empty) virtual CD drive.

2. How full is your root file system?

`/dev/vda2` has 62 GB, 14 GB are used and 45 GB are free: it is 24% full. (Used + available is a bit less than the size, because ext4 reserves 5% of the blocks for root.)

3. How many CPU cores and how much RAM does your laptop have, according to `/proc`?

`/proc/cpuinfo` lists 4 processors, so the system sees 4 cores. `MemTotal` is 4 010 236 kB, i.e. about 3.8 GiB (4 GB assigned to the machine, minus the memory the kernel reserves for itself).

4. `/proc/self/status` showed the name of a program. Which one, and why that one?

It showed `Name: head`. `/proc/self` is a magic link that always points to the directory of *the process that is reading it*. The file was opened by `head`, not by the shell, so `self` meant the `head` process (PID 5123). For the same reason `ls -l /proc/self/fd` showed the descriptors of `ls` itself, including descriptor 3, the directory it was listing.

6 STRETCH TASKS

6.1 (a) Magic Numbers

```
$ gzip -c /etc/passwd > notes.txt ; file notes.txt ; xxd notes.txt | head -1
notes.txt: gzip compressed data, was "passwd", last modified: Tue Sep 30 21:14:02 2026, from Unix, original
    size modulo 2^32 2914
00000000: 1f8b 0808 4a1d dc6a 0003 7061 7373 7764  ....J..j..passwd
```

Listing 14: The type is in the content, not in the name

Although the file is called `notes.txt`, `file` recognises gzip data. It does not look at the extension: it compares the first bytes with a database of signatures. The first two bytes `1f 8b` are the gzip magic number; `08` is the compression method (deflate) and the next `08` is the flag "original file name present", which is why the name `passwd` is visible right after the header.

6.2 (c) Deleting a File That Is Still Open

```
$ head -c 200M /dev/zero > bigfile ; df -h . | tail -1
/dev/vda2        62G   15G   44G  25% /
$ exec 3< bigfile ; rm bigfile ; ls -l /proc/$$/fd
total 0
lrwx------ 1 sergiu sergiu 64 Oct 25 18:35 0 -> /dev/pts/0
lrwx------ 1 sergiu sergiu 64 Oct 25 18:35 1 -> /dev/pts/0
lrwx------ 1 sergiu sergiu 64 Oct 25 18:35 2 -> /dev/pts/0
lr-x------ 1 sergiu sergiu 64 Oct 25 18:35 3 -> /home/sergiu/os-lab5/bigfile (deleted)
lrwx------ 1 sergiu sergiu 64 Oct 25 18:35 255 -> /dev/pts/0
$ df -h . | tail -1
/dev/vda2        62G   15G   44G  25% /
$ exec 3<&- ; df -h . | tail -1
/dev/vda2        62G   14G   45G  24% /
```

Listing 15: A deleted but still open file keeps its space

After `rm` the name is gone and the link count of the inode is 0, but the shell still holds descriptor 3 on it, so the link in `/proc` shows the old path with the suffix `(deleted)`. The 200 MB are still counted as used. The space is really freed only when the last descriptor is closed with `exec 3<&-`: this is exactly the rule "link count = 0 **and** no open references".

CONCLUSIONS

In this laboratory work I examined the Linux file system using only standard commands. I saw that the first letter of `ls -l` identifies the file type and that even `/bin` is, on a modern Ubuntu, just a symbolic link. The experiments with links made the separation between names and inodes very concrete: `rm` only removes a directory entry, a hard link keeps the data alive, a symbolic link is a tiny file that contains a path and breaks when that path disappears, and directories collect one link from every subdirectory's `..` I also saw that the size of a file and the space it occupies are different things, thanks to block allocation and sparse files.

The permission experiments showed that the kernel applies exactly one class of bits, that ownership only grants the right to change permissions, that `r` and `x` mean "list" and "traverse" on a directory, and how the umask shapes every new file. The setuid bit of `passwd` and the sticky bit of `/tmp` explained how ordinary users can safely perform privileged or shared operations. Finally, `lsblk`, `df` and `/proc` showed how disks, partitions and even the kernel's own state are presented as parts of the same single tree. These observations prepare the ground for Laboratory Work No. 6, where the same ideas are studied inside the source code of the xv6 file system.